

☁ CLOUDASTICK SYSTEMS — SALESFORCE PARTNER

ABC Pharmacy Salesforce Solution

Technical Documentation & Architecture Decision Record

PROJECT

ABC Pharmacy Order Management System

API VERSION

67.0 — Winter '26

PREPARED BY

Abdelrhman Ehab Abdelkhalek

DATE

August 2026

ORG INSTANCE

orgfarm-7e74891727-dev-ed

ASSESSMENT

Salesforce Entry — Cloudastick Systems

Table of Contents

Solution Overview

Data Model & Design Decisions

Entity Relationship Diagram

Objects Reference

Key Design Decisions & Rationale

Apex Components

REST API — Product List

REST API — Shopping Cart

Error Logger

Batch Archival Class

Visualforce Controller

Test Classes

Salesforce Features Used

API Reference

Org Access

1. Solution Overview

ABC Pharmacy required a Salesforce environment to manage its product catalog, customer registry, and order lifecycle — integrated with an existing **Ionic web application** used by end-users for shopping, and a **warehouse interface** used by fulfilment staff to manage outbound deliveries.

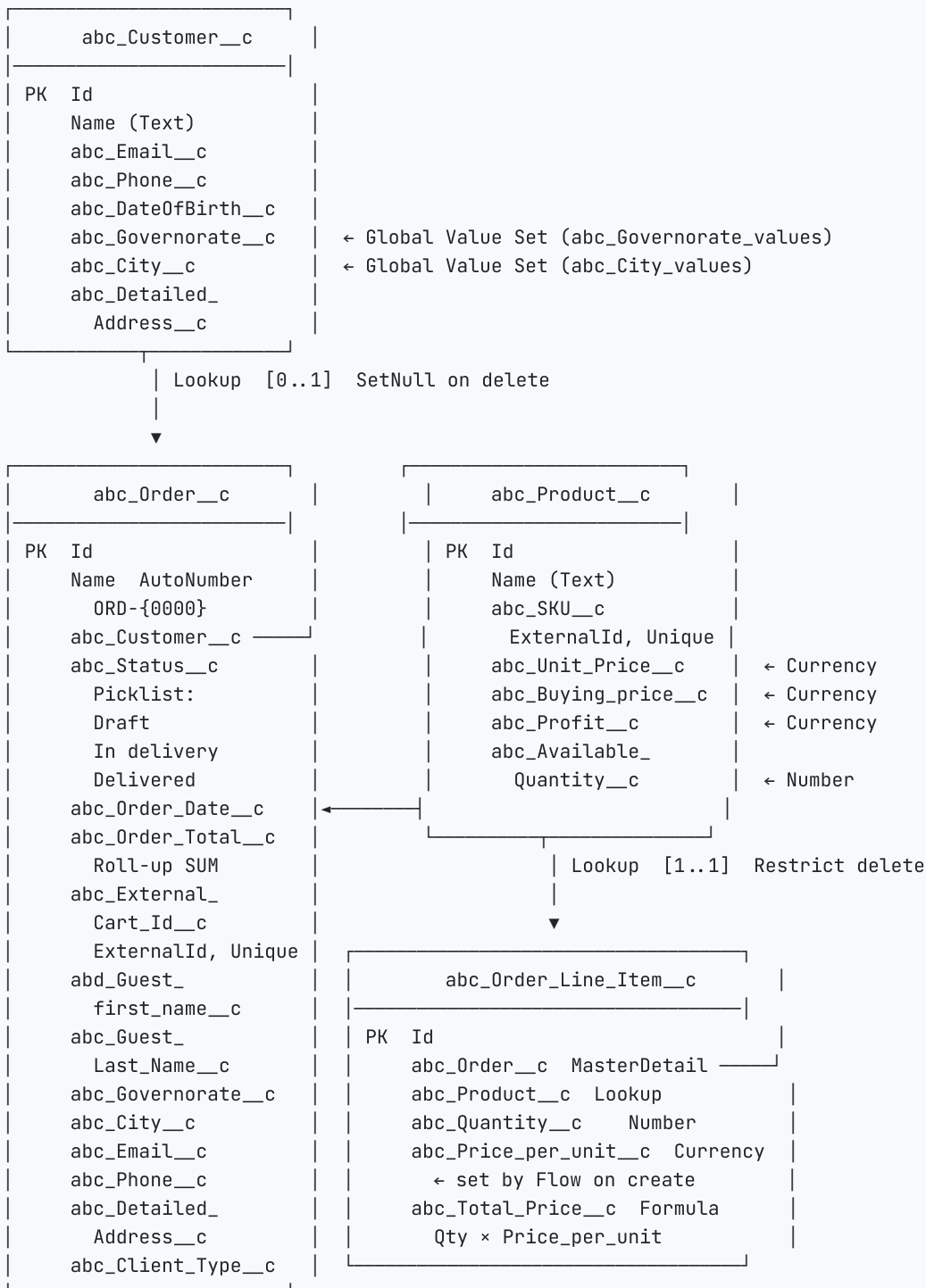
Architecture at a Glance

Layer	Component	Audience
Data	5 Custom Objects + 1 Big Object + 1 Log Object	All users
Automation	Record-Triggered Flow (price freeze), Roll-up Summary (order total)	Internal
Integration	REST API — GET /products, POST /cart	Ionic web app
UI — Internal	Lightning App + Custom Lightning Record Pages	Admin, Pharmacist
UI — Warehouse	Visualforce Page (abc_WarehouseOrders)	Warehouse user
Archival	Batch Apex → Big Object	Scheduled / Admin-triggered
Observability	abc_API_Log__c (structured error records)	Admin
Security	Custom Profiles (pharmacist, warehouse) + FLS	All users

Convention: All custom metadata follows the `abc_` API name prefix as required, ensuring clear namespace separation from standard Salesforce objects and future managed packages.

2. Data Model & Design Decisions

2.1 Entity Relationship Diagram



abc_Archived_Order__b (Big Object – Index: Order Date DESC, Order Number ASC)		
abc_Order_Date__c	DateTime	[Index 1]
abc_Order_Number__c	Text	[Index 2]
abc_Status__c	Text	
abc_Order_Total__c	Number	
abc_Customer_Email__c	Email	
abc_External_Cart_Id__c	Text	
abc_Governorate__c	Text	
abc_City__c	Text	
abc_Client_Type__c	Text	
abc_Detailed_Address__c	Text	

abc_API_Log__c		
Name	AutoNumber	LOG-{{0000}}
abc_Class_Name__c	Text	
abc_Method_Name__c	Text	
abc_Error_Message__c	LongTextArea	
abc_Stack_Trace__c	LongTextArea	

2.2 Objects Reference

Object	Type	Purpose
abc_Customer__c	Custom Object	Registered pharmacy customer profiles with address, contact, and demographics
abc_Product__c	Custom Object	Pharmacy product catalog with pricing (unit/buying/profit) and stock levels
abc_Order__c	Custom Object	Pharmacy orders supporting both registered and guest users via the Ionic app
abc_Order_Line_Item__c	Custom Object	Individual ordered items within an order (Master-Detail → Order, Lookup → Product)
abc_Archived_Order__b	Big Object	Immutable long-term archive of delivered orders older than 1 year
abc_API_Log__c	Custom Object	Structured, queryable error log written by REST API classes on exception

2.3 Key Design Decisions & Rationale

Decision 1 — Custom Objects vs. Standard Salesforce Objects

DECISION

Built five custom objects with the `abc_` prefix rather than repurposing standard objects (Contact → Customer, Product2 → Product, Opportunity → Order).

ALTERNATIVE CONSIDERED

Using standard objects (Contact, Product2, Opportunity + OpportunityLineItem) would have required fewer object licences and some built-in features like Pricebooks.

WHY THIS CHOICE

Standard objects carry implicit business meaning (e.g. Opportunity implies a sales pipeline, not a completed order), complex lifecycle rules, and fields irrelevant to pharmacy operations. Custom objects give a clean domain model that maps directly to pharmacy concepts, avoids polluting the standard CRM with unrelated records, and makes permissions simpler — we grant access only to the objects this business needs.

Decision 2 — Nullable Lookup vs. Required Master-Detail for Customer → Order

DECISION

`abc_Order__c.abc_Customer__c` is a **nullable Lookup** (SetNull on delete) rather than a required Master-Detail or a required Lookup.

ALTERNATIVE CONSIDERED

A required Lookup or Master-Detail would enforce that every order belongs to a customer. This is simpler but breaks the guest checkout use case.

WHY THIS CHOICE

The Ionic app supports **guest checkout** — users can place orders without registering. Forcing a Customer record would require creating a ghost customer for every guest, leading to data pollution. Instead, guest orders store contact details directly on the Order

(`abd_Guest_first_name__c` , `abc_Guest_Last_Name__c` , `abc_Email__c`). SetNull on customer delete preserves order history without cascading deletions.

TRADE-OFF ACCEPTED

Guest and registered orders require conditional logic when displaying customer details. This is handled in both the Visualforce controller and the API — an acceptable complexity given the business requirement for guest checkout.

Decision 3 — Master-Detail vs. Lookup for Line Item → Order

DECISION

`abc_Order_Line_Item__c.abc_Order__c` is a **Master-Detail** relationship to `abc_Order__c` .

WHY THIS CHOICE

Master-Detail enables three critical features at zero extra code cost: (1) **Roll-up Summary** on the Order for the real-time total, (2) **cascade delete** so that deleting an order automatically deletes all its line items, and (3) enforced referential integrity — a line item cannot exist without a parent order. A Lookup would require Apex triggers to replicate these behaviours.

Decision 4 — Before-Save Flow vs. Apex Trigger for Price Freezing

DECISION

Price freezing (copying `Product.abc_Unit_Price__c` → `LineItem.abc_Price_per_unit__c`) is handled by a **before-save Record-Triggered Flow**.

Approach	Transaction Cost	Admin Maintainable	Retroactive Change Risk
✓ Before-save Flow	No extra DML	Yes ✓	None (frozen on create)
After-save Apex Trigger	Extra DML (update)	No	None
Formula Field	None	Yes	High — always reflects current price
Default Field Value	None	Yes	None — but can't reference Product field

WHY THIS CHOICE

A before-save Flow runs in the *same transaction* as the insert, requires no additional DML statement (unlike an after-save trigger that would need an update), and can be modified by a Salesforce admin without code deployment. A formula field was rejected because it would always show the *current* product price — meaning historical orders would retroactively reflect price changes, which is incorrect for financial records.

Decision 5 — Big Object vs. Custom Object for Archiving

DECISION

Delivered orders older than 1 year are archived to `abc_Archived_Order__b` (a Big Object) rather than to a standard custom object.

Approach	Storage Cost	Queryable	Scalability
✓ Big Object	Included (no limit)	Via SOQL with index	Petabyte-scale
Custom Object	Counts against 10GB default	Full SOQL	Limited by storage plan
External database	External cost	Via callout only	Unlimited

WHY THIS CHOICE

A pharmacy accumulating years of order history will generate hundreds of thousands of records. Standard custom object storage (10GB default) would eventually be exhausted, requiring costly storage upgrades. Big Objects are designed precisely for this use case: high-volume, append-only, historically queryable data at no additional storage cost. The compound index (`Order Date DESC` , `Order Number ASC`) ensures efficient retrieval of recent archives.

Decision 6 — Custom REST API vs. Standard Salesforce API for Ionic Integration

DECISION

Two custom `@RestResource` Apex classes rather than exposing the Ionic app to the standard Salesforce REST/SOAP API directly.

WHY THIS CHOICE

The standard Salesforce REST API would require the Ionic app to: (1) know internal Salesforce record IDs (not SKU codes), (2) make multiple round-trips to create an order and its line items, (3) implement idempotency logic itself, and (4) understand Salesforce-specific error structures. A custom REST API abstracts all of this: it accepts SKU codes, creates both Order and Line Items in a single atomic request, handles idempotency via `externalCartId` , and

returns clean business-friendly JSON responses with standard HTTP status codes (201, 400, 404, 422, 500).

Decision 7 — without sharing on WarehouseOrdersController

DECISION

`abc_WarehouseOrdersController` is declared `without sharing`.

WHY THIS CHOICE

The warehouse user needs to see *all* orders regardless of record ownership. Orders are created by the Ionic API (running as a connected app user) or by pharmacists, not by the warehouse user. With `with sharing`, the warehouse profile would only see orders they own — which would be none. Using `without sharing` is appropriate here because access to this specific page is already controlled by profile-level page access permissions (only the warehouse profile can access `/apex/abc_WarehouseOrders`).

Decision 8 — abc_API_Log__c vs. Platform Events vs. Debug Logs

DECISION

API errors are persisted to `abc_API_Log__c` records (custom object).

Approach	Persistent?	Admin Visible?	Queryable?	Complexity
✓ <code>abc_API_Log__c</code> (Custom Object)	Yes	Yes	Yes (SOQL)	Low
System.debug (Debug Logs)	Until purged (2MB limit)	Dev only	No	None
Platform Events	72h (standard)	Needs subscriber	Via subscription	Medium
External monitoring (e.g. Datadog)	Yes	External only	Yes	High

WHY THIS CHOICE

Debug logs require an active debug session to capture and are capped at 2MB — they are a development tool, not a production monitoring solution. Platform Events require a separate

subscriber process. A custom object is the simplest approach that gives admins a permanent, queryable record of all API failures directly in Salesforce — no additional tooling required.

3. Apex Components

3.1 abc_ProductAPI — Product List Endpoint

Type	<code>@RestResource</code> · <code>global with sharing</code>
URL	<code>/services/apexrest/abc/products</code>
Method	GET
Purpose	Returns all Pharmacy Products with <code>abc_Available_Quantity__c > 0</code> , sorted alphabetically. Filters out-of-stock items so the Ionic app never presents products users cannot order.
Error Handling	Full try/catch — exceptions are logged via <code>abc_APILogger</code> and returned as <code>{ "success": false, "error": "..."} </code> with HTTP 500.
Sharing	<code>with sharing</code> — respects FLS and object permissions of the calling user.

3.2 abc_CartAPI — Shopping Cart Endpoint

Type	<code>@RestResource</code> · <code>global with sharing</code>
URL	<code>/services/apexrest/abc/cart</code>
Method	POST
Purpose	Creates <code>abc_Order__c</code> + one <code>abc_Order_Line_Item__c</code> per item in a single atomic operation. Accepts Salesforce IDs or SKU codes as product references.
Idempotency	Checks <code>abc_External_Cart_Id__c</code> before inserting. Duplicate cart IDs return HTTP 200 with the existing order instead of creating duplicates.
Validation	HTTP 400 (empty items / blank productId / zero quantity / invalid customerId) · HTTP 404 (product not found) · HTTP 422 (stock insufficient)
Phone Handling	<code>abc_Phone__c</code> is stored as a Number field in the org. The API accepts the phone as a string, strips non-numeric characters, and converts before assignment.

3.3 abc_APILogger — Error Logger

Type	<code>public without sharing</code>
------	-------------------------------------

Purpose	Centralised error logger called by all API classes. Inserts an <code>abc_API_Log__c</code> record with class name, method, message, and full stack trace.
Resilience	Wraps its own DML in a try/catch and falls back to <code>System.debug</code> if the log insert fails — ensuring the API never crashes because of a logging failure.
Why without sharing	Logs must always be written regardless of the calling user's sharing access. Restricting by sharing rules could silently drop error records.

3.4 abc_ArchiveOrdersBatch — Archival Batch Class

Type	<code>public with sharing</code> · <code>implements Database.Batchable<SObject></code>
Query Filter	<code>abc_Status__c = 'Delivered' AND abc_Order_Date__c < TODAY - 1 year</code>
Write Method	<code>Database.insertImmediate()</code> — required for Big Object DML
Batch Size	200 (default, configurable at execution time)
Email Logic	Prefers linked <code>abc_Customer__r.abc_Email__c</code> ; falls back to the order-level <code>abc_Email__c</code> for guest orders.
Note	Archives are copies — source records are not deleted. Deletion is a separate, explicit business decision.
Run via	<code>Database.executeBatch(new abc_ArchiveOrdersBatch(), 200);</code>

3.5 abc_WarehouseOrdersController — Visualforce Controller

Type	<code>public without sharing</code>
Purpose	Backs <code>abc_WarehouseOrders.page</code> . Loads all orders with their line items in a single SOQL query (sub-query on the Master-Detail relationship).
OrderWrapper	Inner class pairing each order record with a resolved icon (🕒 / ⌚ / 🚚 / ✅) and its list of line items. Status icons take priority over date icons.
Icon Logic	Delivered → ✅ · In delivery → 🚚 · Order Date = Today → ⌚ · Order Date < Today → 🕒
Row Expansion	Client-side JavaScript toggle — no server round-trip, instant UX.

3.6 Test Classes

Test Class	Covers	Scenarios Tested
abc_ProductAPITest	abc_ProductAPI	Returns only in-stock products; empty catalog returns 200 with count 0
abc_CartAPITest	abc_CartAPI	Success by SF ID; success by SKU; idempotency (200 on retry); 400 for empty items; 400 for blank productId; 400 for invalid customerId; 404 for unknown product; 422 for over-stock request
abc_ArchiveOrdersBatchTest	abc_ArchiveOrdersBatch	Old delivered orders are archived; recent delivered orders excluded; draft orders excluded; customer email preferred over order-level email
abc_WarehouseOrdersControllerTest	abc_WarehouseOrdersController	All 4 icon types independently tested; line items loaded correctly; all orders returned

4. Salesforce Features Used

Feature	Component	Why Chosen
Custom Lightning App	<code>abc_PharmaD</code>	Provides a branded navigation space for pharmacists and admins, surfacing only the pharmacy-relevant tabs (Products, Customers, Orders, Line Items). Keeps the pharmacy domain separate from standard Salesforce Sales/Service apps.
Custom Objects & Fields	6 objects, 30+ fields	Clean domain model with pharmacy-specific semantics. Field types chosen deliberately: AutoNumber for Order Name (human-readable IDs), Roll-up Summary for totals (no Apex needed), Formula for line item totals, Currency for prices.
Big Object	<code>abc_Archived_Order__b</code>	Designed for unlimited-scale append-only historical data. Prevents storage exhaustion as order history grows year over year. <code>Database.insertImmediate()</code> writes bypass standard DML governor limits.
Record-Triggered Flow (before-save)	<code>abc_product_and_line_item_price_match</code>	Freezes unit price at order time without extra DML (before-save runs in the same transaction). Admin-maintainable without code deployments. Chosen over Apex trigger for lower maintenance burden.
Roll-Up Summary Field	<code>abc_Order_Total__c</code>	Real-time, always-accurate order total with zero Apex code. Only possible because Line Item has a Master-Detail to Order — this was a deliberate data model choice to unlock this free feature.

Feature	Component	Why Chosen
REST Apex (@RestResource)	abc_ProductAPI, abc_CartAPI	Required for Ionic app integration. Custom REST preferred over standard object API because the cart creation requires multi-step atomic logic (stock validation + parent/child DML + idempotency) that a single standard API call cannot express.
Batch Apex (Database.Batchable)	abc_ArchiveOrdersBatch	Standard Salesforce pattern for large-scale, scheduled data processing. Processes records in configurable chunks (200 default) to stay within governor limits. Cannot be replaced by a Scheduled Flow because Flows cannot call <code>Database.insertImmediate()</code> for Big Objects.
Visualforce Page	abc_WarehouseOrders	Specifically required by the assessment. Server-side icon resolution in Apex controller (clean separation of logic from markup) combined with client-side JavaScript for row expansion (instant UX, no server round-trip).
Custom Profiles	pharmacist, warehouse	Principle of least privilege. Warehouse users get read-only order/product access and can only view the Visualforce page. Pharmacist users manage the full order lifecycle. Role separation mirrors real pharmacy operations.
Lightning Record Pages (Flexipages)	4 custom record pages	Activated on all custom objects to provide a polished Lightning Experience UI beyond the default page layout. Allows drag-and-drop component arrangement without code.
Global Value Sets	abc_Governorate_values, abc_City_values	Egyptian governorates and cities are shared across <code>abc_Customer__c</code> and <code>abc_Order__c</code> . Centralising in Global Value Sets means adding a new city requires one update, not one per object. Ensures

Feature	Component	Why Chosen
		consistency across all picklists referencing the same geography.
External ID Fields	abc_SKU__c, abc_External_Cart_Id__c	SKU lets the Ionic app reference products by real-world codes instead of opaque Salesforce IDs. External Cart ID enables idempotency — the API checks this field before inserting so retried requests do not create duplicate orders. Both fields are indexed for efficient lookups.
Custom WebLinks (Buttons)	abc_Warehouse_Orders_Page, abc_Warehouse_Orders_List	Allows the warehouse user to navigate to the Visualforce page directly from the Order record detail page and from the Orders list view — without needing to know the page URL.
Error Logging Object	abc_API_Log__c	Provides a persistent, admin-visible, SOQL-queryable audit trail of API failures. Chosen over debug logs (transient, dev-only) and Platform Events (requires a subscriber process).

5. API Reference

Authentication

All endpoints require a valid Salesforce OAuth2 Bearer token.

```
Authorization: Bearer <access_token>
```

For production server-to-server integration, the **Connected App + JWT Bearer Flow** is recommended. For testing, the Username-Password flow works with a Connected App.

5.1 GET /services/apexrest/abc/products

Returns all pharmacy products with available stock (`abc_Available_Quantity__c > 0`), sorted alphabetically by name.

Request

```
GET https://<instance-url>/services/apexrest/abc/products
Authorization: Bearer <token>
```

Response 200 — Success

```
{
  "success": true,
  "count": 2,
  "data": [
    {
      "id": "a01bm000001XxxxAAA",
      "name": "Amoxicillin 250mg",
      "sku": "SKU-AMOX-250",
      "unitPrice": 35.0,
      "availableQuantity": 100
    },
    {
      "id": "a01bm000001XyyyAAA",
      "name": "Paracetamol 500mg",
      "sku": "SKU-PARA-500",
      "unitPrice": 15.0,
      "availableQuantity": 200
    }
  ]
}
```

```
]
}
```

Response 500 — Error

```
{ "success": false, "error": "<exception message>" }
```

5.2 POST /services/apexrest/abc/cart

Creates a pharmacy order with one or more line items atomically.

Request

```
POST https://<instance-url>/services/apexrest/abc/cart
Authorization: Bearer <token>
Content-Type: application/json

{
  "externalCartId": "CART-IONIC-001",
  "customerId": "003bm000001XxxxAAA",
  "guestFirstName": "Ahmed",
  "guestLastName": "Ali",
  "governorate": "Cairo",
  "city": "Nasr City",
  "email": "ahmed@example.com",
  "phone": "01001234567",
  "detailedAddress": "123 Tahrir Street, Apt 5",
  "items": [
    { "productId": "SKU-PARA-500", "quantity": 2 },
    { "productId": "a01bm000001XxxxAAA", "quantity": 1 }
  ]
}
```

Request Field Reference

Field	Type	Required	Notes
<code>externalCartId</code>	String	No	Enables idempotency — safe to re-send on retry
<code>customerId</code>	String	No	Salesforce ID of <code>abc_Customer__c</code> ; omit for guest orders
<code>guestFirstName</code>	String	No	Used when no <code>customerId</code> is provided
<code>guestLastName</code>	String	No	
<code>governorate</code>	String	No	Must match <code>abc_Governorate_values</code> picklist

Field	Type	Required	Notes
city	String	No	Must match abc_City_values picklist
email	String	No	Order contact email
phone	String	No	Numeric digits — non-numeric characters are stripped automatically
detailedAddress	String	No	Free text address
items	Array	Yes	Minimum 1 item
items[].productId	String	Yes	Salesforce record ID or abc_SKU_c value
items[].quantity	Integer	Yes	Must be > 0

Response Codes

Status	Condition	Body
201 Created	Order created successfully	{ "success": true, "orderId": "...", "orderNumber": "ORD-0001" }
200 OK	Idempotent — cart already exists	{ "success": true, "message": "Cart already exists.", "orderId": "...", "orderNumber": "..." }
400 Bad Request	Validation failure (empty items, blank productId, zero quantity, invalid customerId)	{ "success": false, "error": "..." }
404 Not Found	Product not found by ID or SKU	{ "success": false, "error": "Product not found: SKU-X" }
422 Unprocessable	Requested quantity exceeds available stock	{ "success": false, "error": "Insufficient stock for product: ... (available: X, requested: Y)" }
500 Server Error	Unexpected exception (logged to abc_API_Log_c)	{ "success": false, "error": "..." }

6. Org Access

Instance URL

```
https://orgfarm-7e74891727-dev-ed.develop.my.salesforce.com
```

Reviewer Account (System Administrator)

Username	assessments@cloudastick.com.pharmaD
Email	assessments@cloudastick.com
Profile	System Administrator
License	Salesforce

Note: Use the "Forgot Password" flow at the instance URL above to set the password for this account before logging in.

Warehouse User

Profile	warehouse
Visualforce Page	/apex/abc_WarehouseOrders
Access via Order record	"Warehouse Orders View" button on any Pharmacy Order record
Access via list view	"Warehouse Orders View" in the Orders list view actions dropdown

Postman Collection

The file `ABC_Pharmacy_Postman_Collection.json` is included in the project root. Import it into Postman, run the **Auth** → **Get Access Token** request first, then test both API endpoints from the Products and Cart folders.

